



中山大學
SUN YAT-SEN UNIVERSITY

本科生课程报告

架构设计文档

寻迹校园失物招领平台

课程名称： 软件工程

项目名称： 寻迹校园失物招领平台

团队名称： 寻迹项目组

项目负责人： 朱梓涵

团队成员： 朱梓涵、周星辰、林洪宽、郭焜华、程文韬

负责人学号： 24325356

文档版本： V1.0

基线日期： 2026 年 7 月 12 日

中山大学

2026 年 7 月

目录

1	文档目的与架构基线	1
2	系统上下文与边界	1
3	容器职责与部署单元	3
4	主后端模块与分层	3
5	数据架构	5
6	关键时序：发布、任务与匹配	7
7	事务、并发与状态一致性	7
8	AI 隔离、降级与可靠性	8
9	安全与隐私架构	9
10	部署与运行架构	10
11	质量属性分析	10
12	关键架构决策记录	11
13	课程目标完成与扩展路线	12
14	结论	13

1 文档目的与架构基线

本文给出「寻迹」校园失物招领平台的整体架构说明。课程项目围绕两个 Vue 前端、FastAPI 主后端、独立 AI 服务、MySQL、MinIO 与容器化部署形成完整技术体系，覆盖系统边界、模块职责、数据组织、核心流程、关键决策与部署方案。架构设计以校园失物招领业务闭环为中心，同时兼顾可维护性、可靠性、安全性和持续演进能力。

表 1: 架构说明范围

说明范围	核心内容	设计重点
系统边界	用户端、管理端、主后端、AI 与外部服务	参与者、容器和通信协议
数据与状态	核心实体、约束、索引和状态机	数据一致性与业务闭环
关键流程	发布、匹配、认领、交接与通知	事务边界与异步协作
工程实现	分层模块、DTO、持久任务与安全机制	低耦合、可测试和可维护
运行拓扑	容器、端口、健康检查和配置	可复现部署与服务治理

1.1 架构目标与驱动因素

平台面向高校内「信息分散、人工检索效率低、认领核验与隐私保护要求高」的问题，核心闭环为「发布—检索与匹配—分级验证—双确认交接—归档与信用记录」。课程项目版本优先保证可演示、可测试、可部署和可追踪，而不是追求互联网规模。主要架构驱动如下：

- 五人团队和课程迭代周期要求降低分布式协调、运维和联调成本；
- AI 能力具有不稳定、耗时和依赖较重的特征，不能成为 P0 主流程的单点故障；
- 认领、交接、信用和治理操作涉及多实体一致性，需要明确事务与并发守卫；
- 证件、凭证和图片需要严格保护，存储、签名、展示与 AI 获取采用分层授权；
- 课程交付覆盖需求、接口、实现、测试和部署的完整软件工程过程。

2 系统上下文与边界

2.1 参与者与外部依赖

表 2:系统参与者与协作关系

参与者/系统	主要目标	与寻迹平台的交互
普通用户	找回失物或归还拾获物	注册登录、发布、搜索、查看匹配、发起认领、确认交接、接收通知
后勤/安保人员	批量登记和代管拾获物	以 STAFF 身份发布招领、参与审核与交接
系统管理员	治理与运营	认证和内容审核、举报处置、公告、用户状态、统计和操作日志查询
MinIO	保存非结构化资产	保存图片和凭证；bucket 保持私有，后端按权限动态签发读取 URL
AI 服务	提供辅助计算	分类、文本匹配、敏感检测；不直接连接业务数据库
DashScope/兼容模型服务	增强智能推理	由 AI 服务统一调用，并配合确定性规则保证稳定输出

课程项目将学校统一身份、商业短信、支付、物流和实时聊天定义为外部扩展边界。当前版本采用受白名单约束的验证码演示路径，并把 AI 模型设计为可替换增强能力，保证基础发布和认领流程始终独立、稳定。

2.2 系统容器图

图 1 展示运行时容器及主要协议。两个前端镜像均采用多阶段构建：Node 构建 Vue 静态资源，Nginx 提供 SPA 回退并将相对路径 /api/ 代理到主后端。

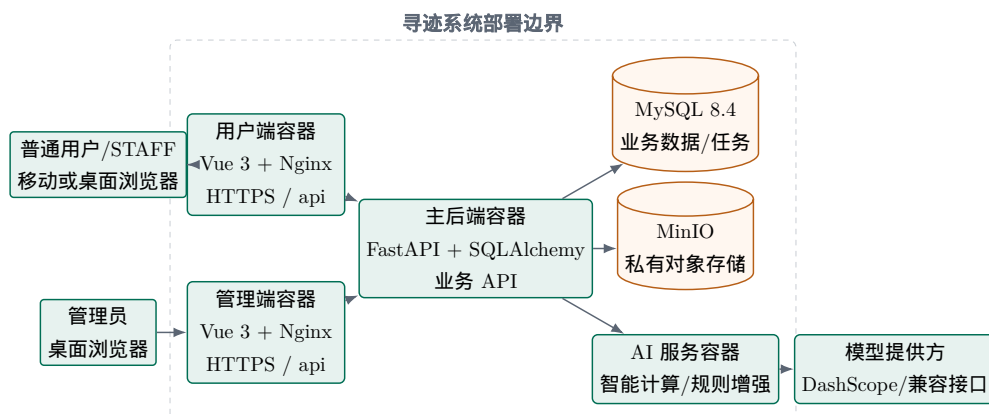


图 1: 寻迹系统容器与外部依赖

3 容器职责与部署单元

表 3:部署容器职责

容器	技术与入口	关键职责	状态/持久性
user-web	Vue 3、Pinia、Vue Router、Element Plus、Nginx	用户发布、搜索、匹配、认领、通知与个人中心	前端无业务真相；token 位于浏览器存储
admin-web	Vue 3、Pinia、Vue Router、Element Plus、Nginx	审核、举报、公告、用户、统计和日志界面	权限最终由后端判断
backend	Python 3.12、FastAPI、SQLAlchemy async、Alembic	业务规则、鉴权、事务、对象签名、任务 runner、统一响应	业务状态写入 MySQL
ai-service	Python 3.12、FastAPI、httpx、Pillow	分类、敏感检测、文本/规则评分	无业务数据库、无业务持久化
mysql	MySQL 8.4	业务实体、日志、通知、积分、持久任务	命名卷 mysql-data
minio	固定日期版本 MinIO 镜像	图片和凭证对象；禁止匿名 bucket policy	命名卷 minio-data

默认 Compose 将 MySQL 与 MinIO 端口绑定到 127.0.0.1，生产覆盖文件进一步移除二者的宿主机端口。后端容器启动命令先执行 alembic upgrade head，再启动 Uvicorn。两个前端通过相对 API 路径复用同源代理，减少浏览器跨域配置。健康检查目前主要证明进程可响应，并不等价于数据库、对象存储和业务就绪。

4 主后端模块与分层

4.1 模块职责

表 4: 主后端模块划分

模块	核心职责	主要协作关系
core	配置、JWT、异常、AI HTTP 客户端、对象存储、 管理员引导	为所有业务域提供基础设施
common/db	响应包、错误码、分页、校 验、时间、Base、 AsyncSession、ULID	无业务规则的共享能力
user	注册登录、资料、认证、账 号状态	认证审批、积分、通知、注销级联
item	失物/招领、搜索、图片、验 证问题、举报提交	发布后写 job；为 match/claim 提供 物品视图
job	持久任务/outbox、领取、执 行、重试、回收	驱动分类、敏感检测和匹配
match	候选遍历、评分、结果 upsert、匹配通知	调用 AI；为 claim 提供关联上下文
claim	分级验证、审核、申诉、交 接和结案	编排 item、match、credit、 notification、log
credit	积分变更、边界裁剪、流水 幂等	在调用方事务中写用户、流水、通知 与日志
notification	站内通知、未读数、公告公 开查询	被匹配、认领、积分、治理模块调用
admin	认证/内容审核、举报处置、 公告、用户、看板	跨业务域编排和操作审计
operation_ log	关键动作的审计记录与查询	与业务状态在同一会话提交

4.2 分层约束与模块协作

设计约束规定每个业务包采用 router/service/repository/models/schemas/deps：router 负责 HTTP、依赖和 Schema；service 负责状态守卫、事务和跨域编排；repository 只负责 SQLAlchemy 查询与写入；ORM 与 Pydantic DTO 分离。当前所有

业务 router 均通过 deps 获得 service，未直接导入 repository 或 DB session，这是已实现的清晰边界。

跨模块协作由 service 统一编排，user/service.py、item/service.py、match/service.py 负责组合相邻领域能力；repository 和 model 只承担明确的数据访问职责。该组织方式在保持模块内聚的同时，支持匹配、认领和交接等跨域事务完整落地。

4.3 模块与持久任务关系图

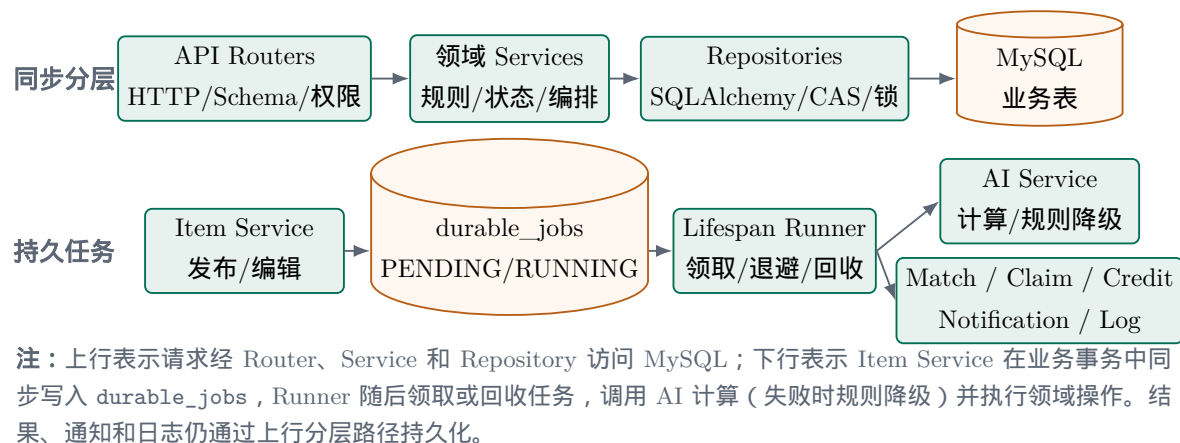


图 2: 分层调用与持久任务协作关系

5 数据架构

5.1 聚合与实体关系

核心实体关系以用户、物品、匹配和认领为主轴：用户可发布多条 lost/found；一条 found 可包含多张图片和最多三个验证问题；lost 与 found 的候选关系由唯一 match pair 表示；claim 可选关联 match，且一条 claim 最多一个 handover；通知、积分、举报和操作日志记录横切副作用。所有业务主键为 26 位 ULID，避免依赖单库自增并保持大体时间有序。

表 5: 核心数据表与一致性作用

表组	代表表	关键约束/用途
用户与认证	users、user_cert_requests	手机号唯一；用户状态、角色、认证和信用快照
物品与资产	lost_items、found_items、item_images、verify_questions	状态、审核、敏感标记；文件仅存稳定引用
匹配	match_results	lost_item_id + found_item_id 唯一；保存分项分、来源与降级标志
认领与交接	claim_requests、claim_answers、handover_records	handover 的 claim 唯一；回答保留题目快照
信用与通知	credit_logs、notifications	积分按用户、业务、原因唯一；通知带关联类型与 ID
治理与审计	reports、announcements、operation_logs	同一举报人对同一精确目标唯一；关键状态动作可追溯
可靠任务	durable_jobs	类型、业务、版本唯一；状态/领取与业务版本索引

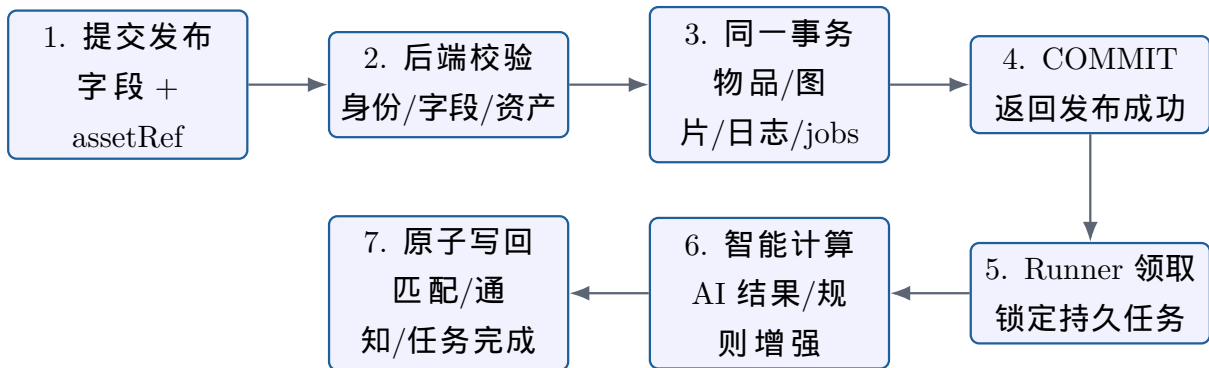
5.2 索引、约束与迁移

数据库结构以 Alembic 为唯一演进链，当前单一 head 为 20260711_0008。迁移不仅创建表，还包含历史数据收敛：0004 在增加 match pair 唯一约束前重写 claim 和 notification 引用并删除重复行；0007 在举报唯一约束前清理重复；0008 增加带检查约束、领取索引和版本唯一键的持久任务表。部署不再挂载历史手写 backend/sql/schema.sql，旧库必须先结构比对再决定迁移或 stamp。

业务关系采用逻辑 ID 和应用层守卫，在保持课程迭代低耦合的同时，由 service 统一维护引用完整性；部署过程通过数据核对、明确删除策略和逐步增加外键持续强化数据库约束。普通枚举主要由 Schema/service 校验，同时 durable_jobs 对类型和状态设置了数据库检查约束。

6 关键时序：发布、任务与匹配

图 3 对应已落地的可靠发布路径。接口成功只表示物品与任务已经提交，不表示 AI 已完成；分类失败会重试，匹配在 AI 不可用时可直接使用规则评分，敏感检测未得出明确安全结论时保持保护状态。



Runner 使用行锁领取任务；AI 响应不可用时切换为规则评分，执行异常时按退避策略重试。

图 3: 物品发布与持久任务执行流程

6.1 匹配算法与可解释性

综合评分权重为图像 0.4、文本 0.3、地点 0.2、时间 0.1，但只在「双方均有输入且该维度确有可用评分器」的维度上归一。主后端不信任远端 totalScore，会校验所有分值、来源、可用性并重新计算。课程版本以文本、地点与时间评分为稳定基线，图像维度通过 imageAvailable=false 显式关闭；规则版使用中英文词元 Jaccard、地点相等/包含关系和失物时间区间距离。总分低于 70 不保留活动匹配，达到 90 且候选为证件时发高优先级通知。数据库同时保存 degraded 与 scoreSource，使前端和运营可以区分模型增强与规则结果。

7 事务、并发与状态一致性

7.1 事务所有权

repository 只执行查询、flush 或条件更新，不自行 commit；顶层 service 复用一個 AsyncSession 编排多表写入并在末尾统一 commit。发布流程在一次提交中写物品、图片、操作日志和 job；任务执行在一次提交中写匹配、通知和任务完成；交接结案在一次提交中写 claim、found、关联 lost、handover、积分、通知和日志。异常离开请求时未提交会话由依赖生命周期关闭并回滚。

设计文档要求使用显式 async with session.begin()，但现有 service 大多依赖

SQLAlchemy 的 `autobegin` 后统一 `commit()`。其原子性来自同一会话的一次提交，代码形态与书面硬约束并不完全一致；后续应统一事务模板并增加失败注入测试。

7.2 并发控制手段

表 6: 并发与幂等控制

场景	已实现机制	目的
同一招领被并发认领	锁定 <code>found/claim/match</code> ; <code>found</code> 使用期望状态 <code>CAS</code> : <code>PENDING</code> 到 <code>CLAIMING</code>	只有一个申请占用物品
并发审核	行锁加源状态条件更新	状态变化后返回冲突，不覆盖新结果
双方并发确认交接	锁定 <code>claim</code> 、 <code>found</code> 、 <code>handover</code> ; 确认位更新后重新读取	只在双方均确认时结案一次
重复交接记录	<code>handover</code> 的 <code>claim</code> 唯一键，加 <code>savepoint</code> 捕获并发冲突	重复创建返回已有记录
重复积分	业务唯一键、用户行锁、 <code>savepoint</code> 、边界裁剪	状态与积分保持一致且可重试
重复匹配任务	<code>match pair</code> 唯一、任务业务版本唯一、 <code>upsert</code>	不重复创建 <code>pair</code> 与通知
多 runner 领取任务	<code>FOR UPDATE SKIP LOCKED</code> 、 <code>RUNNING</code> 租约、版本检查	并行领取且进程崩溃后可回收

核心状态以 `docs/architecture/enums.md` 为唯一字典：`lost` 从 `SEARCHING` 到 `FOUND` 或 `CLOSED`；`found` 从 `PENDING` 到 `CLAIMING`，再到 `RETURNED` 或 `CLOSED`；`claim` 包含 `PENDING`、`ANSWER_PASSED`、`PROOF_PENDING`、`APPROVED`、`REJECTED`、`APPEALING`、`HANDED_OVER` 和 `TERMINATED`。状态变更由 `service` 校验角色、所有权和源状态，前端只展示后端返回的能力和结果。

8 AI 隔离、降级与可靠性

AI 服务是独立进程，通过 `/internal/ai/*` 提供分类、敏感检测和匹配评分。主后端使用带 3 秒默认超时和 `X-Service-Token` 的 `httpx.AsyncClient`；响应必须满足键

集合、类型、有限数值、取值范围和来源白名单，否则按失败处理。AI 服务自身可调用 DashScope 原生或兼容接口，并复用进程级连接池。

表 7: AI 能力与降级策略

能力	正常路径	降级/失败路径
物品分类	VL 模型返回五类之一，关键词补充标签	关键词分类，标记 degraded=true；无可用结果的持久任务重试
文本匹配	聊天评分或 embedding 余弦相似度	中文二元/英文词元 Jaccard，加地点与时间规则
图像相似度	设计预留真实模型维度	当前明确不可用，分值为 0，不用「有图」伪造相似度
敏感检测	VL 返回唯一类别或 NONE	任意超时、空结果、降级、需复核均保持敏感；任务退避重试

这种策略体现「业务可用性优先、隐私安全从严」：AI 失败不回滚已经提交的基础发布，但也不能将未证明安全的图片公开。当前敏感检测不生成可信脱敏副本，因此无权用户看到空图片而非伪造的「脱敏图」。

9 安全与隐私架构

9.1 身份、权限与配置安全

用户 JWT 解析后仍会查询数据库，只有存在且状态为 ACTIVE 的用户可继续访问，因而旧 token 不能绕过账号禁用；管理员接口使用后端角色依赖，不依赖前端路由守卫。密码采用 PBKDF2-SHA256、随机盐和恒时比较。非 DEBUG 环境若仍使用仓库默认 JWT 或管理员密码，后端拒绝启动；CORS 不允许通配符，只接受显式 origin 列表。AI 内部接口除健康检查外均要求至少 32 字符服务令牌，使用恒时比较；空令牌只在明确的本地开发模式允许。

9.2 图片与对象存储安全

上传服务不信任 MIME、文件名或扩展名，而是使用 Pillow 解码、限制 10 MiB 和像素数、拒绝 decompression bomb，再按真实 JPEG/PNG/WebP 格式重编码，以去除

EXIF 和文本元数据。对象 key 包含业务类型、上传者、年月和随机 UUID；提交业务时只接受当前用户、对应业务类型的 `asset://` 引用，并通过 MinIO stat 验证对象存在。

MinIO bucket 在启动和上传时确保存在并删除匿名 policy。普通与敏感资源使用不同签名时长；AI 收到的是由内部 MinIO endpoint 独立生成的短签名，不是浏览器预览 URL。AI 下载器还执行协议/凭据/主机白名单检查、DNS 解析后的私网检查、每次重定向复验、大小限制、内容类型与实际解码格式比对，防范 SSRF 与恶意图片请求。

10 部署与运行架构

10.1 环境划分

表 8: 环境与部署策略

环境	推荐形态	关键约束
本地开发	MySQL/MinIO 容器，前后端源码热更新；也可完整 Compose	允许受控 debug SMS 和 mock/规则 AI，不提交真实秘密
小组联调	固定主机和 API，应用容器化，数据库与对象存储地址稳定	固定枚举和字段；使用独立测试账号与数据
课程演示	复用已演练联调环境，冻结代码、迁移和演示数据	答辩前不临时改表；保留规则降级和固定脚本
扩展部署	HTTPS 网关，仅公开前端/443，内部化 backend/AI/MinIO	可按实际运行环境继续增强

配置通过 Pydantic Settings 和环境变量注入；示例文件进入仓库，真实 `.env` 被忽略。Compose 对数据库、JWT、管理员和 MinIO 密码使用必填展开，AI token 在共享环境应替换为唯一随机值。镜像安装均使用锁文件和 frozen 模式，前端、后端、AI 均有独立 Dockerfile。

11 质量属性分析

表 9: 质量属性与架构响应

属性	已采用的设计	实现效果
可维护性	业务域模块化、 router/service/repository、 DTO/ORM 分离、文档化枚举和 状态机	模块职责清晰，接口与数据结构 可独立演进
可靠性	DB outbox、任务租约、退避、版本去重、AI 规则降级、fail-closed	异步任务可恢复，外部模型波动 不阻塞主流程
一致性	单会话统一提交、行锁、CAS、 唯一约束、savepoint	多实体状态在统一事务中闭环， 重复请求保持幂等
安全性	JWT 状态复核、角色依赖、私有 存储、短签名、SSRF 防护、默认 密钥拒启	身份、对象和服务调用均采用分 层授权
性能	全异步 HTTP/DB、连接池、受 控 AI 并发、索引与分页	发布请求快速返回，耗时计算由 持久任务异步处理
可部署性	锁定依赖、Alembic、Compose、 健康检查、多阶段镜像	开发、联调和课程演示环境可重 复构建
可审计性	统一响应、业务错误码、操作日 志、积分流水、匹配来源	关键业务动作与状态变化均可追 踪

12 关键架构决策记录

表 10: 架构决策摘要

ADR	决策	原因与收益	演进条件
001	主后端采用模块化 单体，AI 独立部 署	控制五人团队的联调、配置 和排错成本，同时隔离模型 依赖	业务模块共享数据库； 团队/流量扩大后再按 边界拆分
002	主后端与 AI 统一 Python async 栈	FastAPI/Pydantic 契约清晰， httpx 与异步 DB 适合外部调 用	CPU/阻塞任务需线程 池或独立 worker

ADR	决策	原因与收益	演进条件
003	MySQL + 私有 MinIO，库中只存稳定引用	结构化数据与大对象分离，可动态授权和缩短敏感 URL	需要资产绑定、孤儿清理和对对象权限运维
004	数据库 outbox 替代仅内存 BackgroundTasks	发布与任务原子落库，进程重启可恢复；无需引入消息中间件	runner 与应用同进程，规模扩大应迁移独立 worker/队列
005	AI 可降级，敏感判断 fail-closed	模型不可用不阻断主流程，又不以不确定结果换取公开展示	结果质量受规则限制，需记录来源并支持人工复核
006	双 Vue SPA 共享 TypeScript 类型	用户端移动优先、管理端桌面任务密集，复用枚举和契约	共享包目前是源码别名，需加强契约自动生成
007	ULID 与统一枚举	跨模块 ID 一致、可排序；避免中文状态和魔法值扩散	持续增强数据库与接口枚举一致性校验

13 课程目标完成与扩展路线

本项目已经完成课程范围内的整体架构目标：双前端覆盖用户与管理场景，模块化主后端承载核心业务，独立 AI 服务提供智能增强，MySQL 与 MinIO 分别管理结构化数据和文件资产，持久任务机制串联分类、敏感检测、匹配与通知，Docker Compose 和持续集成支撑统一交付。

在课程成果基础上，系统可沿以下方向继续扩展：

1. 引入校园统一身份与商业短信服务，进一步丰富真实校园环境的身份接入方式；
2. 增加图像向量、候选召回索引与批量评分，提升大规模物品匹配效率；
3. 将 runner 扩展为独立 worker，并结合队列、指标和管理页面强化任务运营；
4. 增加统一 HTTPS 网关、集中观测、备份恢复和不可变镜像，支持更高等级部署；
5. 持续自动生成接口契约和架构约束检查，保持文档、枚举与实现同步演进。

14 结论

寻迹架构以模块化单体承载强一致业务，以独立 AI 服务承载可替换计算，以 MySQL outbox 保证异步任务可恢复，并通过私有对象存储、动态签名和 fail-closed 保护图片资产。系统实现了「发布不被 AI 阻塞、状态变化可追踪、多实体写入统一提交、敏感结果分层展示」四项核心原则，完整支撑校园失物招领从发布到交接归档的课程项目目标。